Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 4

Disciplina: Tehnici de programare

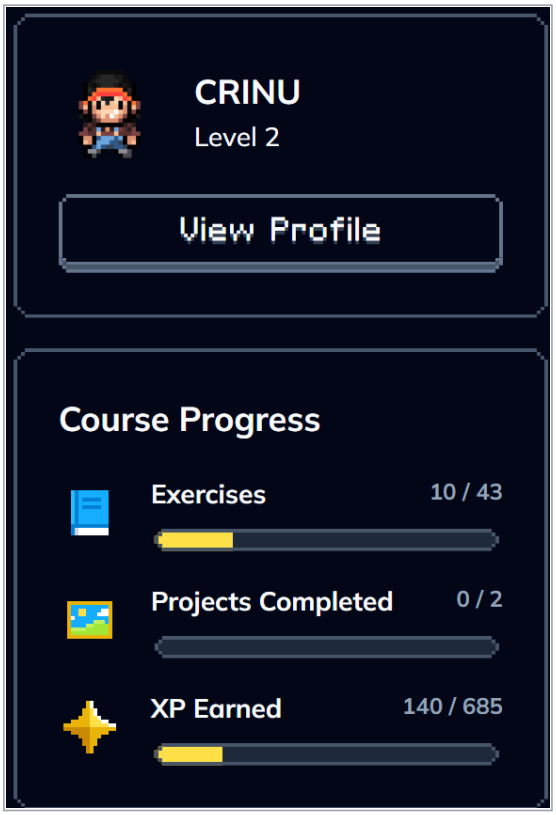
Tema: ” Tehnici de programare in Python ”

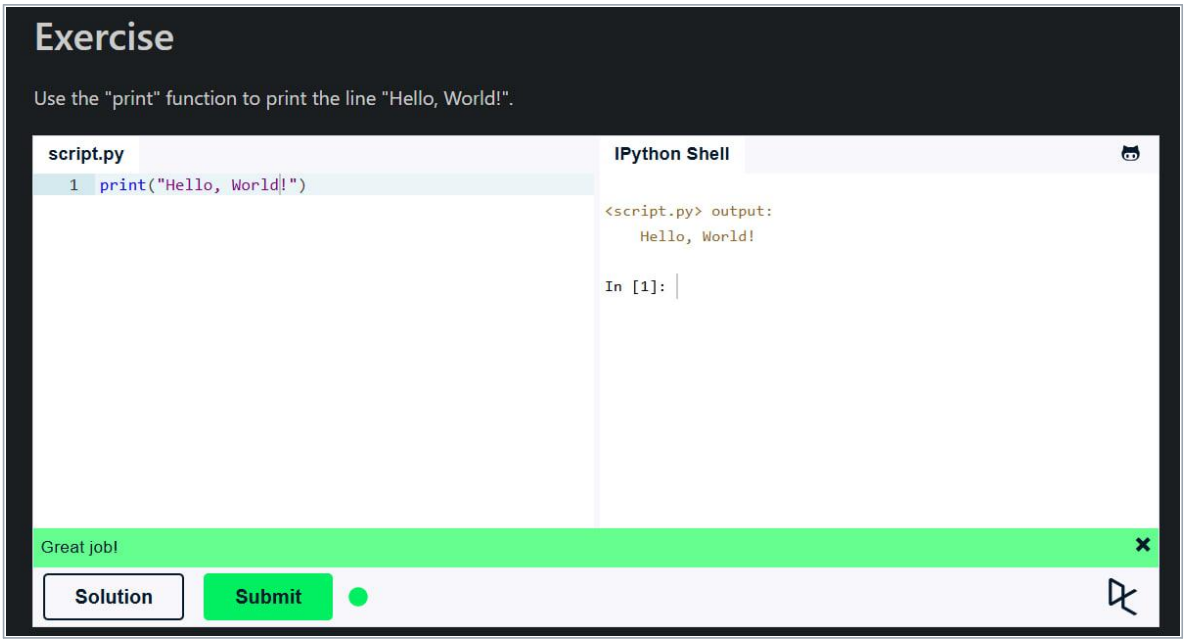
A efectuat : st. gr.SI – 263 Cibric Crinu

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII





Exercise

In this exercise, you will need to add numbers and strings to the correct lists using the "append" list method. You must add the numbers 1,2, and 3 to the "numbers" list, and the words 'hello' and 'world' to the strings variable.

You will also have to fill in the variable second_name with the second name in the names list, using the brackets operator `[]`. Note that the index is zero-based, so if you want to access the second item in the list, its index will be 1.

script.py

```
numbers.append(1)
numbers.append(2)
numbers.append(3)
strings = []
strings.append("hello")
strings.append("world")
names = ["John", "Eric", "Jessica"]

# write your code here
second_name = names[1]


# this code should write out the filled arrays and
the second name in the names list (Eric).
print(numbers)
print(strings)
```

IPython Shell

```
<script.py> output:
    [1, 2, 3]
    ['hello', 'world']
    The second name on the names list is Eric

In [1]:
```

Great Job!

Solution Submit

Exercise

You will need to write a format string which prints out the data using the following syntax: `Hello John Doe. Your current balance is $53.44.`

script.py

```
data = ("John", "Doe", 53.44)
format_string = "Hello %s %s. Your current balance
is $%s."

print(format_string % data)
```

IPython Shell

```
<script.py> output:
    Hello John Doe. Your current balance is $53.44

<script.py> output:
    Hello John Doe. Your current balance is $53.44.

In [1]:
```

Great work!

Solution Submit

Exercise

Change the variables in the first section, so that each if statement resolves as True.

script.py

```
6
7  if number > 15:
8      print("1")
9
10 if first_array:
11     print("2")
12
13 if len(second_array) == 2:
14     print("3")
15
16 if len(first_array) + len(second_array) == 5:
17     print("4")
18
19 if first_array and first_array[0] == 1:
20     print("5")
21
```

IPython Shell

```
<script.py> output:
    1
    2
    3
    4
    5
    6

In [1]:
```

Great Work!

Solution | Submit

Exercise

The target of this exercise is to create a string, an integer, and a floating point number. The string should be named `mystring` and should contain the word "hello". The floating point number should be named `myfloat` and should contain the number 10.0, and the integer should be named `myint` and should contain the number 20.

script.py

```
1  # change this code
2  mystring = "hello"
3  myfloat = 10.0
4  myint = 20
5
6  # testing code
7  if mystring == "hello":
8      print("String: %s" % mystring)
9  if isinstance(myfloat, float) and myfloat == 10.0:
10     print("Float: %f" % myfloat)
11 if isinstance(myint, int) and myint == 20:
12     print("Integer: %d" % myint)
```

IPython Shell

```
<script.py> output:
    String: hello
    Float: 10.000000
    Integer: 20

<script.py> output:
    String: hello
    Float: 10.000000
    Integer: 20

In [1]:
```

Great job!

Solution | Submit

Exercise

The target of this exercise is to create two lists called `x_list` and `y_list`, which contain 10 instances of the variables `x` and `y`, respectively. You are also required to create a list called `big_list`, which contains the variables `x` and `y`, 10 times each, by concatenating the two lists you have created.

script.py

```
y = object()

# TODO: change this code
x_list = [x] * 10
y_list = [y] * 10
big_list = x_list + y_list

print("x_list contains %d objects" % len(x_list))
print("y_list contains %d objects" % len(y_list))
print("big_list contains %d objects" % len(big_list
))

# testing code
if x_list.count(x) == 10 and y_list.count(y) == 10:
    print("Almost there...")
if big_list.count(x) == 10 and big_list.count(y) ==
```

IPython Shell

```
<script.py> output:
    x_list contains 10 objects
    y_list contains 10 objects
    big_list contains 20 objects
    Almost there...
    Great!

In [1]:
```

Good work!

Solution | Submit

Exercise

Try to fix the code to print out the correct information by changing the string.

script.py

```
# Convert everything to uppercase
print("String in uppercase: %s" % s.upper())

# Convert everything to lowercase
print("String in lowercase: %s" % s.lower())

# Check how a string starts
if s.startswith("Str"):
    print("String starts with 'Str'. Good!")

# Check how a string ends
if s.endswith("ome!"):
    print("String ends with 'ome!'. Good!")

# Split the string into three separate strings,
# each containing only a word
```

IPython Shell

```
Length of s = 20
The first occurrence of the letter a = 8
a occurs 2 times
The first five characters are 'Strin'
The next five characters are 'gs ar'
The thirteenth character is 'a'
The characters with odd index are 'tig r wsm!'
The last five characters are 'some!'
String in uppercase: STRINGS ARE AWESOME!
String in lowercase: strings are awesome!
String starts with 'Str'. Good!
String ends with 'ome!'. Good!
Split the words of the string: ['Strings', 'are',
    'awesome!']
```

Great work!

Solution | Submit

Exercise

Loop through and print out all even numbers from the numbers list in the same order they are received. Don't print any numbers that come after 237 in the sequence.

script.py

```
    399, 162, 758, 219, 918, 237, 412, 566, 826,
248, 866, 950, 626, 949, 687, 217,
    815, 67, 104, 58, 512, 24, 892, 894, 767, 553,
81, 379, 843, 831, 445, 742, 717,
    958, 609, 842, 451, 688, 753, 854, 685, 93, 857
, 440, 380, 126, 721, 328, 753, 470,
    743, 527
]

# your code goes here
for number in numbers:
    if number == 237:
        break

    if number % 2 == 1:
        continue
```

IPython Shell

```
File "script.py", line 13
  if numbers < 237
               ^
SyntaxError: invalid syntax

<script.py> output:
    402
    984
    360
    408
    980
    544
    390
    984
```

Great work!

Solution | Submit

Exercise

In this exercise you'll use an existing function, and while adding your own to create a fully functional program.

1. Add a function named `list_benefits()` that returns the following list of strings: "More organized code", "More readable code", "Easier code reuse", "Allowing programmers to share and connect code together"
2. Add a function named `build_sentence(info)` which receives a single argument containing a string and returns a sentence starting with the given string and ending with the string " is a benefit of functions!"
3. Run and see all the functions work together!

script.py

```
def list_benefits():
    return "More organized code", "More readable
code", "Easier code reuse", "Allowing programmers
to share and connect code together"

# Modify this function to concatenate to each
benefit - " is a benefit of functions!"
def build_sentence(benefit):
    return "%s is a benefit of functions!" %
benefit


def name_the_benefits_of_functions():
    list_of_benefits = list_benefits()
    for benefit in list_of_benefits:
        print(build_sentence(benefit))
```

IPython Shell

```
  ['More organized code', 'More readable code', 'Easier
      code reuse', 'Allowing programmers to share and
      connect code together']
File "script.py", line 21
  name_the_benefits_of_functions()s()
                                  ^
SyntaxError: invalid syntax

<script.py> output:
  More organized code is a benefit of functions!
  More readable code is a benefit of functions!
  Easier code reuse is a benefit of functions!
  Allowing programmers to share and connect code together
      is a benefit of functions!
```

Nice work!

Solution | Submit

Exercise

We have a class defined for vehicles. Create two new vehicles called car1 and car2. Set car1 to be a red convertible worth $60,000.00 with a name of Fer, and car2 to be a blue van named Jump worth $10,000.00.

script.py

```
        return desc_str

# your code goes here
car1 = Vehicle()
car1.name = "Fer"
car1.color = "red"
car1.kind = "convertible"
car1.value = 60000.00

car2 = Vehicle()
car2.name = "Jump"
car2.color = "blue"
car2.kind = "van"
car2.value = 10000.00

# test code
```

IPython Shell

```
    car1.name(mashinaa)
NameError: name 'mashinaa' is not defined
Traceback (most recent call last):
  File "script.py", line 14, in <module>
    car1.name(mashinaa)
NameError: name 'mashinaa' is not defined
Traceback (most recent call last):
  File "script.py", line 14, in <module>
    car1.name(mashinaa)
NameError: name 'mashinaa' is not defined

<script.py> output:
    Fer is a red convertible worth $60000.00.
    Jump is a blue van worth $10000.00.
```

Great job!

Solution | Submit

Exercise

Add "Jake" to the phonebook with the phone number 938273443, and remove Jill from the phonebook.

script.py

```
phonebook = {

    "John" : 938477566,
    "Jack" : 938377264,
    "Jake" : 938273443
}
# your code goes here

# testing code
if "Jake" in phonebook:
    print("Jake is listed in the phonebook.")

if "Jill" not in phonebook:
    print("Jill is not listed in the phonebook.")
```

IPython Shell

```
<script.py> output:
    Jake is listed in the phonebook.
    Jill is not listed in the phonebook.

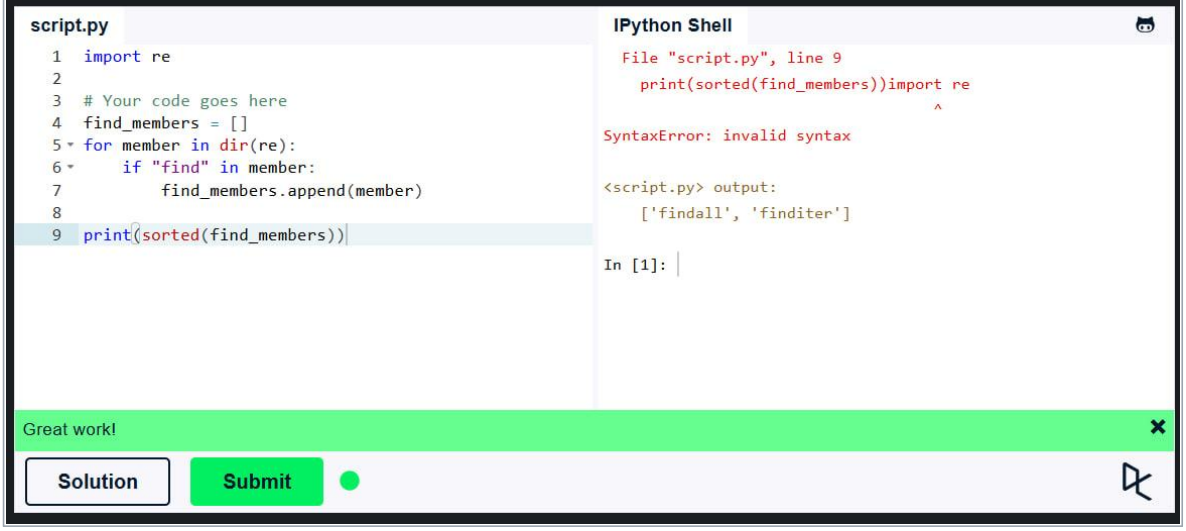
In [1]:
```

Nice work!

Solution | Submit

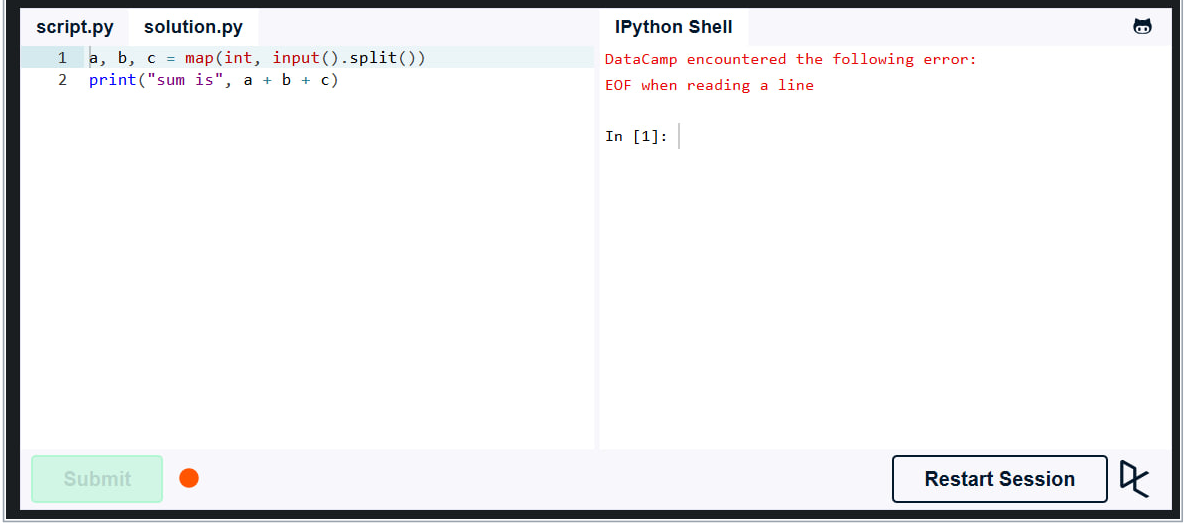
Exercise

In this exercise, print an alphabetically sorted list of all the functions in the `re` module containing the word `find`.



Exercise

Complete the code below so that it reads three integers from a single line, converts each to an `int`, and prints their sum.



Exercise

Write a generator function which returns the Fibonacci series. They are calculated using the following formula: The first two numbers of the series is always equal to 1, and each consecutive number returned is the sum of the last two numbers. Hint: Can you use only two variables in the generator function? Remember that assignments can be done simultaneously. The code

script.py

```
def fib():
    a, b = 1, 1
    while True:
        yield a
        a, b = b, a + b
```

IPython Shell

```
In [1]:
```

Run Restart Session

will simultaneously switch the values of a and b.

script.py

```
# fill in this function
def fib():
    pass
def fib():
    a, b = 1, 1
    while True:
        yield a
        a, b = b, a + b

# testing code
import types
if type(fib()) == types.GeneratorType:
    print("Good, The fib function is a generator.")

    counter = 0
```

IPython Shell

```
<script.py> output:
    Good, The fib function is a generator.
    1
    1
    2
    3
    5
    8
    13
    21
    34
    55
```

Good work!

Solution Submit

Exercise

Write a generator function which returns the Fibonacci series. They are calculated using the following formula: The first two numbers of the series is always equal to 1, and each consecutive number returned is the sum of the last two numbers. Hint: Can you use only two variables in the generator function? Remember that assignments can be done simultaneously. The code

script.py

```
a = 1
b = 2
a, b = b, a
print(a, b)
```

IPython Shell

```
In [1]:
```

Run

will simultaneously switch the values of a and b.

script.py

```
# fill in this function
def fib():
    a, b = 1, 1
    while 1:
        yield a
        a, b = b, a + b

# testing code
import types
if type(fib()) == types.GeneratorType:
    print("Good, The fib function is a generator.")

    counter = 0
    for n in fib():
        print(n)
        counter += 1
```

IPython Shell

```
<script.py> output:
    Good, The fib function is a generator.
    1
    1
    2
    3
    5
    8
    13
    21
    34
    55
```

Good work!

Solution Submit

Exercise

Using a list comprehension, create a new list called "newlist" out of the list "numbers", which contains only the positive numbers from the list, as integers.

script.py

```
1  numbers = [34.6, -203.4, 44.9, 68.3, -12.2, 44.6,
   12.7]
2  newlist = [int(x) for x in numbers if x > 0]
3  print(newlist)
```

IPython Shell

```
<script.py> output:
    [34, 44, 68, 44, 12]

In [1]:
```

Very nice!

Solution | Submit

Exercise

Write a program using lambda functions to check if a number in the given list is odd. Print "True" if the number is odd or "False" if not for each element.

script.py

```
1  l = [2,4,7,3,14,19]
2  for i in l:
3      # your code here
4      my_lambda = lambda x : (x % 2) == 1
5      print(my_lambda(i))
```

IPython Shell

```
<script.py> output:
    False
    False
    True
    True
    False
    True

In [1]:
```

Nice work!

Solution | Submit

Exercise

Fill in the `foo` and `bar` functions so they can receive a variable amount of arguments (3 or more) The `foo` function must return the amount of extra arguments received. The `bar` must return `True` if the argument with the keyword `magicnumber` is worth 7, and `False` otherwise.

script.py

```
# edit the functions prototype and implementation
def foo(a, b, c, *args):
    return len(args)

def bar(a, b, c, **kwargs):
    return kwargs["magicnumber"] == 7


# test code
if foo(1, 2, 3, 4) == 1:
    print("Good.")
if foo(1, 2, 3, 4, 5) == 2:
    print("Better.")
if bar(1, 2, 3, magicnumber=6) == False:
    print("Great.")
if bar(1, 2, 3, magicnumber=7) == True:
```

IPython Shell

```
<script.py> output:
    Good.
    Better.
    Great.
    Awesome!

In [1]:
```

Great work!

Solution | Submit

Regular Expressions

Regular Expressions (sometimes shortened to regexp, regex, or re) are a tool for matching patterns in text. In Python, we have the re module. The applications for regular expressions are wide-spread, but they are fairly complex, so when contemplating using a regex for a certain task, think about alternatives, and come to regexes as a last resort.

An example regex is `r"^(From|To|Cc).*?python-list@python.org"` Now for an explanation: the caret `^` matches text at the beginning of a line. The following group, the part with `(From|To|Cc)` means that the line has to start with one of the words that are separated by the pipe `|`. That is called the OR operator, and the regex will match if the line starts with any of the words in the group. The `.*?` means to un-greedily match any number of characters, except the newline `\n` character. The un-greedy part means to match as few repetitions as possible. The `.` character means any non-newline character, the `*` means to repeat 0 or more times, and the `?` character makes it un-greedy.

So, the following lines would be matched by that regex: `From: python-list@python.org To: !asp]<,. python-list@python.org`

A complete reference for the re syntax is available at the python docs.

As an example of a "proper" email-matching regex (like the one in the exercise), see this

script.py

```
# Exercise: make a regular expression that will
match an email
import re
def test_email(your_pattern):
    pattern = re.compile(your_pattern)
    emails = ["john@example.com", "python
-list@python.org", "wha.t.`1an?ug{}ly@email.com"]
    for email in emails:
        if not re.match(pattern, email):
            print("You failed to match %s" % (email
))
        elif not your_pattern:
            print("Forgot to enter a pattern!")
        else:
            print("Pass")
# Your pattern here!
```

IPython Shell

```
<script.py> output:
    <_sre.SRE_Match object; span=(35, 39), match='[on]'>
    None
    Forgot to enter a pattern!
    Forgot to enter a pattern!
    Forgot to enter a pattern!

<script.py> output:
    Pass
    Pass
    Pass

In [1]:
```

Great work!

Solution | Submit

Exercise

Handle all the exception! Think back to the previous lessons to return the last name of the actor.

script.py

```
1  actor = {"name": "John Cleese", "rank": "awesome"}
2
3  def get_last_name():
4      return actor["name"].split()[1]
5
6  get_last_name()
7  print("All exceptions caught! Good job!")
8  print("The actor's last name is %s" % get_last_name
   ())
```

IPython Shell

```
<script.py> output:
    All exceptions caught! Good job!
    The actor's last name is Cleese

In [1]:
```

Great work!

Solution | Submit

In the exercise below, use the given lists to print out a set containing all the participants from event A which did not attend event B.

script.py

```
1  a = ["Jake", "John", "Eric"]
2  b = ["John", "Jill"]
3
4  A = set(a)
5  B = set(b)
6
7  print(A.difference(B))
```

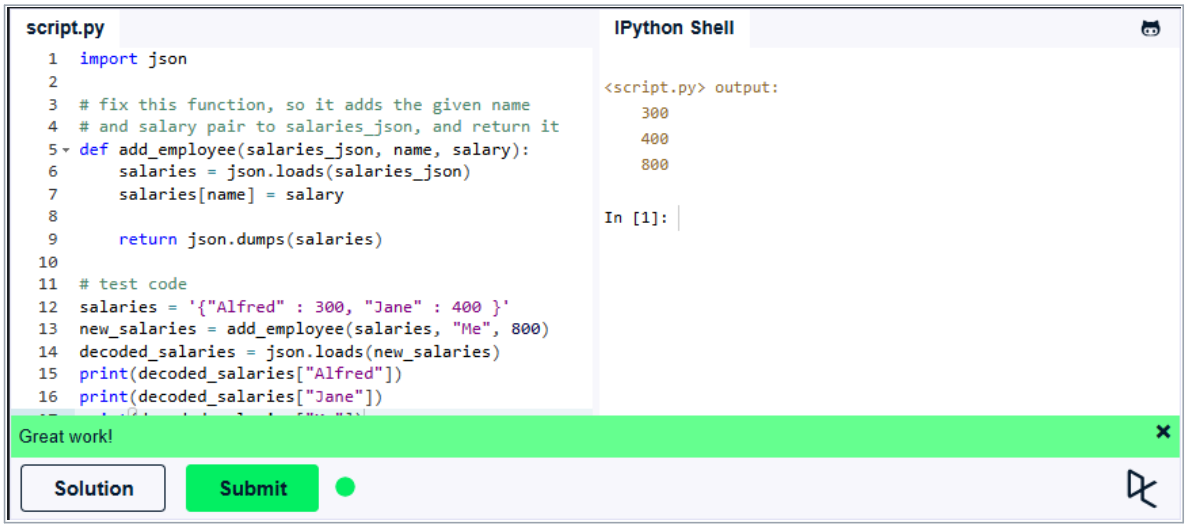
IPython Shell

```
<script.py> output:
    {'Jake', 'Eric'}

In [1]:
```

Nice work!

Solution | Submit


script.py
IPython Shell
1 import json
2
3 # fix this function, so it adds the given name
4 # and salary pair to salaries_json, and return it
5 def add_employee(salaries_json, name, salary):
6 salaries = json.loads(salaries_json)
7 salaries[name] = salary
8
9 return json.dumps(salaries)
10
11 # test code
12 salaries = '{"Alfred" : 300, "Jane" : 400 }'
13 new_salaries = add_employee(salaries, "Me", 800)
14 decoded_salaries = json.loads(new_salaries)
15 print(decoded_salaries["Alfred"])
16 print(decoded_salaries["Jane"])
<script.py> output:
300
400
800
In [1]:
Great work!
Solution
Submit

CONCLUZIE

În cadrul acestei lucrări de laborator am parcurs un set de exerciţii practice de programare în limbajul Python, realizate pe o platformă interactivă online. Acest mod de lucru mi-a permis să-mi consolidez cunoştinţele de bază ale limbajului şi să le aplic imediat în cod funcţional, verificând rezultatul după fiecare exerciţiu.

Am început cu elementele fundamentale: afişarea textului, variabilele şi tipurile de date (şiruri de caractere, numere întregi şi reale), formatarea şi operaţiile cu şiruri. Apoi am lucrat cu structurile de date ale limbajului – liste, dicţionare şi mulţimi (*set*) – şi am folosit instrucţiunile condiţionale şi buclele *for* şi *while*, împreună cu *break* şi *continue*, pentru a controla fluxul de execuţie al programului.

Pe parcurs am studiat şi noţiuni mai avansate: funcţii cu număr variabil de argumente (**args* şi ***kwargs*), funcţii *lambda*, generatoare (seria Fibonacci), *list comprehensions*, clase şi obiecte, expresii regulate (modulul *re*), tratarea excepţiilor şi lucrul cu formatul JSON.

În concluzie, efectuarea acestei lucrări m-a ajutat să înţeleg mai bine sintaxa şi particularităţile limbajului Python şi mi-a arătat cât de concis şi flexibil poate fi codul scris în acest limbaj. Cunoştinţele dobândite îmi vor fi utile la realizarea unor programe mai complexe în cadrul următoarelor lucrări de laborator şi, în general, în activitatea de programare.

BIBLIOGRAFIE

1. Learn Python – Free Interactive Python Tutorial. DataCamp. Disponibil: https://www.learnpython.org/
2. Python Software Foundation. *The Python Tutorial*. Python 3 Documentation. Disponibil: https://docs.python.org/3/tutorial/
3. Python Software Foundation. *re – Regular expression operations*. Python 3 Documentation. Disponibil: https://docs.python.org/3/library/re.html
4. Python Software Foundation. *json – JSON encoder and decoder*. Python 3 Documentation. Disponibil: https://docs.python.org/3/library/json.html